Fitra

[Project Plan]

Version 1.1

Signature Page

Prepared by	**Date**
Gary Quah (Project Manager)	Date: 07/10/2026
Lau Jing Jie (QA Manager)	Date: 07/10/2026
Lee Zheng Wee (QA Engineer)	Date: 07/10/2026
Rusyaidi (QA Engineer)	Date: 07/10/2026
Quah Yew Yi (Release engineer/manager)	Date: 07/10/2026
Matthew Wong (Lead developer)	Date: 07/10/2026
Gary Quah (Project Manager)	Date: 07/10/2026

Revision History

Revision Number	Date	Primary Author(s)	Comments
0.1	Sept 24th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	First edit
0.2	Sept 30th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Second edit
1.0	Oct 7th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Final edit
1.1	Nov 4th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Revised deployment and shared check-in descriptions; updated Function Point assumptions, language gearing, effort/duration, staffing capacity and notional cost calculations (§5). Recorded the capacity shortfall and mitigation assumptions.

Table of Contents

Revision History **1**
Table of Contents **2**
1 Introduction **3**
1.1 Project Overview 3
1.2 Project Description and Scope 3
2 Project Organisation **5**
2.1 Team Structure 5
2.2 Roles and Responsibilities 5
2.3 Team Communication 6
3 Process Definition **7**
3.1 Lifecycle Model 7
4. Schedule **8**
4.1 Activity Dependencies and Schedule 8
4.2 Work Breakdown Structure 11
4.3 Work Packages 12
4.4 Activity Dependencies 12
4.5 Work Package Details 13
5 Project Estimates **19**
5.1 Code Size Estimation using Function Points 19
5.1.1 Unadjusted Function Points 19
5.1.2 Adjusted Function Points 25
5.1.3 Lines of Code 27
5.2 Efforts, Duration and Team Size Estimation 27
5.2.1 Distribution of Effort 28
5.3 Cost Estimates 29
6 Product Checklist **30**
7 Best Practice Checklist **31**
8 Risk Management **33**
9 Quality Assurance **36**
10 Monitoring & Control **37**
11 References **37**

1 Introduction

1.1 Project Overview

The Smart Fitness Tracker is an intelligent, full-stack web application designed to assist users in managing their personal health, fitness and nutrition. By centralising biological health data, manual inputs, and external athletic telemetry, the system leverages Artificial Intelligence (AI) & data analytics to generate dynamic training schedules, automate dietary tracking, and deliver personalised contextual coaching based on daily readiness

1.2 Project Description and Scope

The Smart Fitness Tracker project addresses the fragmentation and high friction found in conventional health-tracking tools by combining human biological metrics, athletic activity, and dietary habits into a single platform. The project encompasses end-to-end development, including the user interface, backend application infrastructure, and external API integrations.

The core of the Smart Fitness Tracker system is an analytics and AI-driven engine that processes multiple inputs to tailor fitness and meal plans. Key system inputs and operational parameters include:

1. Baseline Health Metrics & Goals: User registration data, including personal biometrics, activity levels and fitness goals, used to compute Basal Metabolic Rate (BMR), Total Daily Energy Expenditure (TDEE), and daily macronutrient targets.
2. Activity & Athletic Telemetry: Training and workout history imported via export files from third-party health services, mainly Strava, or manual session logging
3. Subjective Daily Check-Ins: One shared daily questionnaire records 1–5 ratings for muscle fatigue, mood and yesterday's diet adherence. Sleep hours are entered separately on the Training page.
4. Dietary Image Input: User-submitted meal photographs analysed via the Google Gemini API to estimate calories and macronutrient composition.
5. Contextual Coaching Inquiries: Natural language conversational queries regarding progress and training, answered by an AI chatbot referencing the user's historical and expenditure records.

Given these parameters, the system executes an automated readiness model that categorises each day into actionable training states (deload, light, or push) to drive dynamically updated workout recommendations. Concurrently, gamification mechanics—including experience points (XP), dynamic peer leaderboards, interactive daily check-ins, and shared challenges—are implemented to sustain user engagement and retention.

The scope of this project encompasses:

1. **Frontend Web Application:** Development of responsive, browser-based interfaces for input handling, daily check-ins, conversational AI interactions, and the visualisation of training trends, nutrition breakdowns, and readiness scores. The client application runs in standard web browsers without requiring proprietary hardware or dedicated wearable devices.
2. **Backend Application Service: One Python FastAPI service contains application rules, authenticated APIs, profile and metric management, data access and in-process readiness scoring.**
3. **AI & External Integration Modules: Modules within the same FastAPI service call Google Gemini for meal analysis and AI coaching and Strava for account linking and on-demand activity retrieval. These are code modules and external API integrations, not a separate inference deployment.**

Hardware development, medical-grade clinical diagnostics, and direct hardware-level wearable sensor drivers are beyond the scope of this project; wearable devices remain strictly optional for supplementary heart rate data collection. The project team is responsible for delivering the functional web application codebase, database schemas, API integration services, and associated documentation.

2 Project Organisation

The following are the key roles within the project team. Each role is assigned specific responsibilities to ensure effective project management, software development, quality assurance and release management. Team members are expected to collaborate and communicate regularly to ensure project objectives and quality requirements are achieved.

2.1 Team Structure

The following is the list of executive roles.

- Project Manager: Gary Quah
- QA Manager: Lau Jing Jie
- QA Engineers: Lee Zheng Wee, Rusyaidi
- Release Engineer / Manager: Quah Yew Yi
- Lead Developer: Matthew Wong

2.2 Roles and Responsibilities

Project Manager: Gary Quah

- Oversees the overall project and ensures project objectives are met.
- Develops and maintains the project schedule and milestones.
- Coordinates tasks and communication among team members.
- Monitors project progress, resources and risks.
- Reports project status to relevant stakeholders.

QA Manager: Lau Jing Jie

- Oversees software quality assurance activities.
- Develops and maintains the Software Quality Assurance Plan.
- Ensures the project follows defined processes, standards and quality requirements.
- Reviews QA activities and findings.
- Coordinates resolution and follow-up of quality issues.

QA Engineer (Testing & Verification) : Lee Zheng Wee

- Develops and executes software test cases.
- Performs functional and integration testing.
- Identifies, records and tracks software defects.
- Verifies that identified defects have been resolved.
- Reports testing results and findings to the QA Manager

QA Engineer (Process & Documentation) : Rusyaidi

- Reviews project deliverables against defined requirements and quality standards.

- Monitors compliance with established project processes and procedures.
- Conducts reviews and audits of project documentation.
- Maintains QA records, checklists and review findings.
- Supports the QA Manager in monitoring and improving software quality.

Release Engineer / Manager: Quah Yew Yi

- Manages the preparation and release of software builds.
- Maintains release versions and related documentation.
- Coordinates deployment and release activities.
- Ensures releases meet the required quality and configuration criteria.
- Maintains records of released versions.

Lead Developer: Matthew Wong

- Leads software development activities and provides technical direction.
- Coordinates development tasks among developers.
- Reviews source code and technical implementation.
- Ensures development follows agreed coding standards and requirements.
- Works with QA and Release teams to resolve technical issues.

2.3 Team Communication

TLA communication channels include the following:

- Weekly meetings are held on Wednesday.
- Group announcements and updates are sent through Telegram
- Zoom Call discussions are held as necessary.
- Split up into subgroups as necessary, in order to work more co-operatively on specific problems.

3 Process Definition

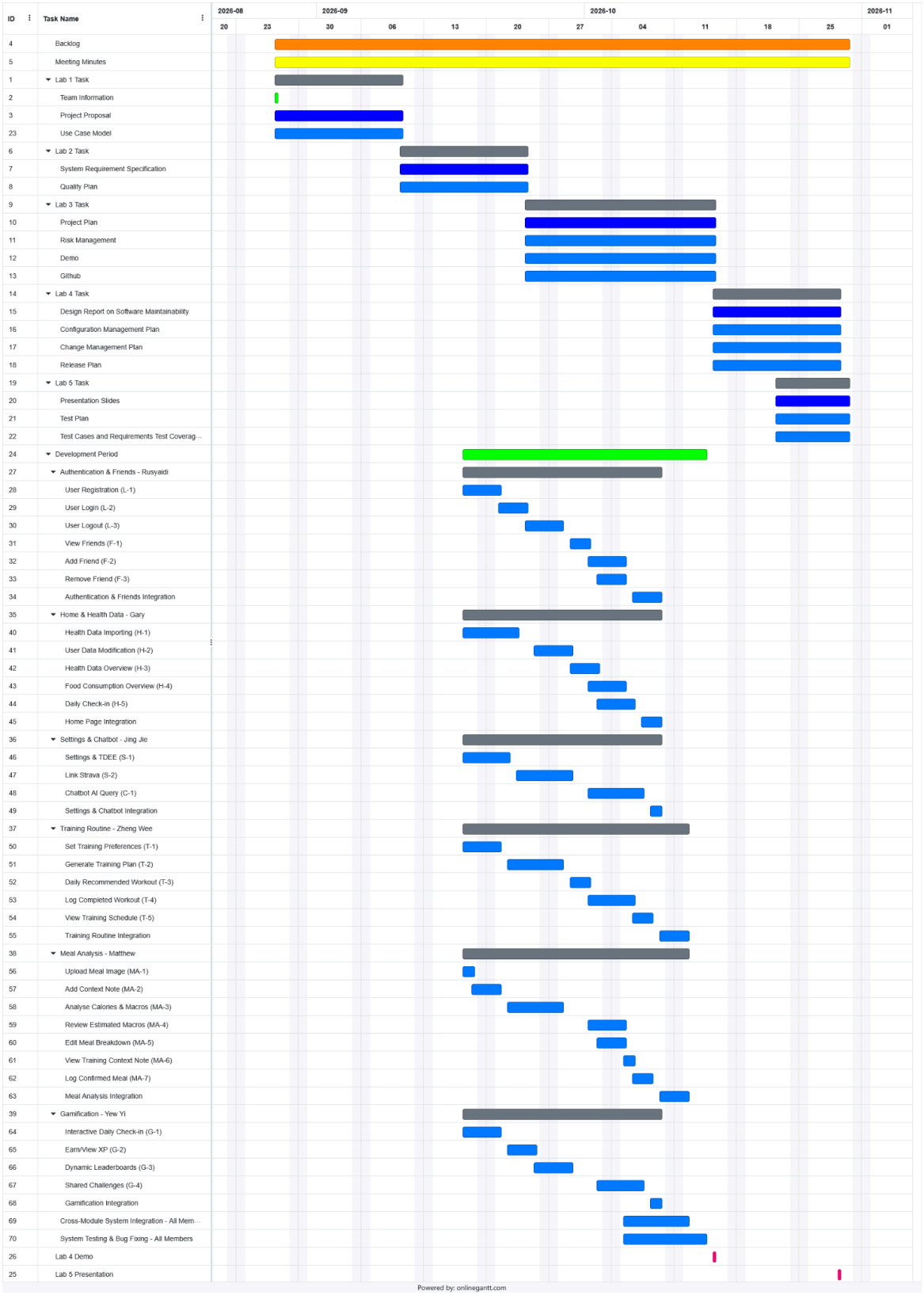
3.1 Lifecycle Model

Fitra will follow the Agile development model throughout the project. This methodology supports iterative and incremental development, allowing the team to continuously develop, test and refine the software based on feedback and changing requirements. The team will work in short development cycles, with regular reviews and collaboration among team members to identify and address issues early.



4. Schedule

4.1 Activity Dependencies and Schedule



Online Gantt 20261104.pdf

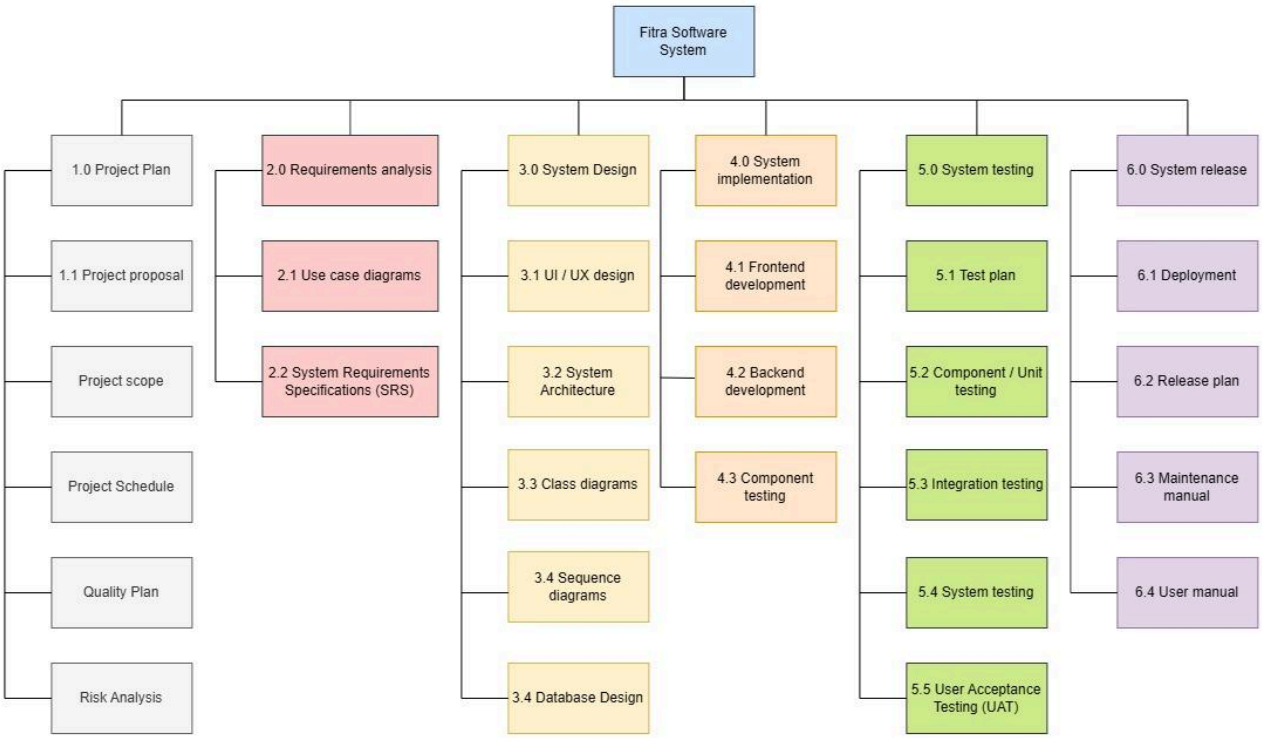
There is a nine-week timeframe to implement a prototype system of a fitness tracker from project commencement on Wednesday of week 3 (27 August 2026) to the end of week 11 (Sunday, 1st November 2026)

Gantt Chart Schedule

Task	Start	End	Duration
Backlog	27 August 2026	30 October 2026	65 Days
Meeting Minutes	27 August 2026	30 October 2026	65 Days
Lab 1 Tasks			
Team Information	27 August 2026	27 August 2026	1 Day
Project Proposal	27 August 2026	10 September 2026	15 Days
Use Case Model	27 August 2026	10 September 2026	15 Days
Lab 2 Tasks			
System Requirement Specification (SRS)	10 September 2026	24 September 2026	15 Days
Quality Plan	10 September 2026	24 September 2026	15 Days
Lab 3 Tasks			
Project Plan	24 September 2026	15 October 2026	22 Days
Risk Management	24 September 2026	15 October 2026	22 Days
Demo (Preparation)	24 September 2026	15 October 2026	22 Days
Github	24 September 2026	15 October 2026	22 Days
Lab 4 Tasks			
Design Report on Software Maintainability	15 October 2026	29 October 2026	15 Days

Configuration Management Plan	15 October 2026	29 October 2026	15 Days
Change Management Plan	15 October 2026	29 October 2026	15 Days
Release Plan	15 October 2026	29 October 2026	15 Days
Lab 5 Task			
Presentation Slides	22 October 2026	29 October 2026	8 Days
Test Plan	29 October 2026	07 November 2026	10 Days
Test Cases and Requirements Test Coverage Report	29 October 2026	04 November 2026	7 Days
Development Period	17 September 2026	15 October 2026	29 Days
Lab 4 Demo	15 October 2026	15 October 2026	1 Day
Lab 5 Presentation	29 October 2026	29 October 2026	1 Day

4.2 Work Breakdown Structure



4.3 Work Packages

The project work is divided into the following work packages to support the Agile development and delivery of Fitra:

1. Project Planning
2. Requirements Analysis and Specification
3. User Interface Design
4. System Architecture and Database Design
5. Software Development
6. Testing and Quality Assurance
7. Integration and Deployment
8. Documentation and Project Review

4.4 Activity Dependencies

The following table describes the dependencies of the deliverable work packages:

Work Package #	Work Package Description	Duration	Dependencies
X01	Project Plan	7 days	--
X02	Requirement Specification	7 days	--
X03	User Interface	7 days	--
X04	Technical Architecture	12.1 days	X01,X02,X03
X05	Data Modeling	7 days	X04
X06	Coding & Unit Testing	16.2 days	X05
X07	Integration & System Testing	16.2 days	X06

The following Activity Network Diagram describes the above in more graphical detail:

4.5 Work Package Details

Work packages are listed below. A team member, indicated in bold, has been assigned as primarily responsible for each work package and will coordinate that package.

Project	**Fitra**
Work Package	X01— Project Plan (1 of 7)
Assigned To	**Jing Jie**, Gary Quah, Zheng Wee, Matthew Wong, Yew Yi, Rusyaidi
Effort	7PD
Start Date	Thursday, 27/08/26
Purpose	To define the overall project scope, lifecycle model, and initial development schedule to establish baseline planning for subsequent deliverables.
Purpose Inputs	Course lab brief, initial problem statements, high-level platform requirements
Activities	This work package includes providing a brief overview of the project, its objectives, and a set of proposed project deliverables throughout the development of the software cycle. The people responsible for this work package will also be transcribing ideas brought up in the group meeting discussion into a formal report. Formulate project objective, assign core roles (PM, QA, Lead Dev, Release Manager, Developers)
Activities Outputs	Formal project plan document, initial sprint backlogs, Lab 1 Meeting Minutes

Project	Fitra
Work Package	X02— Requirement Specification (2 of 7)
Assigned To	**Rusyaidi**, Jing Jie, Gary Quah, Zheng Wee, Matthew Wong, Yew Yi
Effort	7PD
Start Date	Thursday, 09/17/26
Purpose	To establish a clear and verifiable understanding of functional and non-functional requirements, constraints and external interface dependencies
Purpose Inputs	Customer's requirements, Strava API documentation, Gemini
Activities	Conduct object-oriented domain analysis, develop comprehensive UML, use case diagrams and specifications across all core modules.
Activities Outputs	Complete System Requirements Specifications (SRS), and Use Case Models

Project	**Fitra**
Work Package	X03— User Interface (3 of 7)
Assigned To	**Gary Quah,** Rusyaidi, Jing Jie, Zheng Wee, Matthew Wong, Yew Yi
Effort	7PD
Start Date	Thursday, 17/09/26
Purpose	To build and design responsive, accessible web interfaces that facilitate athletic telemetry visualization, meal photo ingestion, interactive daily check-ins, and conversation coaching
Purpose Inputs	SRS, use cases, design mockups

Activities	Design Next.js page layouts, build frontend features and components
Activities Outputs	Next.js frontend client pages

Project	**Fitra**
Work Package	X04— Technical Architecture (4 of 7)
Assigned To	**Gary Quah,** Rusyaidi, Jing Jie, Zheng Wee, Matthew Wong, Yew Yi
Effort	12PD
Start Date	Thursday, 24/09/26
Purpose	To architect full-stack microservice infrastructure, establish secure connection to OAuth, and build API pipelines
Inputs	Project Plan Work Packages (X01 to X03 inclusive).
Activities	High-level design entails defining the architecture of the software system and identifying the various components and how they are inter-related to and interact with each other. Designers also need to decide on the software and hardware infrastructures, such as what operating system on which the software is built, the language used to implement the software, and so on. Design topics including maintainability, portability, and reusability will be addressed here as well
Activities Outputs	High-Level Design and Architectural Specification.

Project	Fitra
Work Package	X05— Data Modeling & Persistence Layer (5 of7)
Assigned To	**Zheng Wee,** Gary Quah, Rusyaidi, Jing Jie, Matthew Wong, Yew Yi
Effort	8PD

Start Date	Thursday, 24/9/26
Purpose	To design and deploy a relational schema, implement security access policies, and establish a training load and nutritional aggregation data pipeline
Purpose Inputs	SRS use case models, class diagrams, and mathematical models (Mifflin-St Jeor)
Activities	Configure Supabase Row-Level Security (RLS) policies, Model relational tables (users, meals, workout_session, daily_checkins, etc)
Activities Outputs	Active Supabase database instances

Project	Fitra
Work Package	X06— Code Feature & Unit testing (6 of 7)
Assigned To	**Matthew Wong,** Zheng Wee, Gary Quah, Rusyaidi, Jing Jie, Yew Yi
Effort	8PD
Start Date	Thursday, 24/9/26
Purpose	To implement core business logic, training plan generators, and unit tests across frontend and backend services
Purpose Inputs	SRS
Activities	Developers will implement the modules according to the design specifications noted in the Specification document.
Activities Outputs	Source code and header files

Project	Fitra

Work Package	X07— Integration & System Testing (7 of 7)
Assigned To	**Yew Yi,** Matthew Wong, Zheng Wee, Gary Quah, Rusyaidi, Jing Jie
Effort	14PD
Start Date	Thursday 15/10/26
Purpose	To conduct end-to-end integration testing, black-box verification, demo readiness and final project touch-up
Purpose Inputs	Test Plan, Integrated codebase
Activities	The Integration testing team may try to simulate how a user might interact with the system. Similar to Unit Testing, Integration Testing may require the development of stubs and drivers as well, but here this is more geared towards the higher (overall system) level. Testers may also examine issues such as system performance and integrity. Heuristics assessment plays an important role in this work package, as intelligence components will define eventual system success.
Activities Outputs	Live Demonstration Prototype, Presentation Deck

5 Project Estimates

5.1 Code Size Estimation using Function Points

We calculated the unadjusted function points based on the complexity of the functions provided by this system. The code size is then estimated from the adjusted function points.

5.1.1 Unadjusted Function Points

The application supports the following proposed functions:

User:

- Register an account, log in and log out
- Enter biological metrics, activity level and fitness goal to obtain BMR, TDEE and macronutrient targets
- Import health and activity data, link a Strava account, or log a session manually
- Submit a daily check-in and receive a daily readiness score
- Set training preferences and generate a multi-week training plan
- View and complete the daily recommended workout
- View, reschedule and recalculate the training schedule
- Log a meal by photograph or manual entry and track it against the daily target
- View training and nutrition trends on the dashboard
- Query the AI assistant about their own training and nutrition history
- Add and remove friends, and view achievements and the leaderboard

The measure of unadjusted function points is based on five primary component elements of these functions: Inputs, Outputs, Inquiries, Logical Files, and Interfaces. Each element ranges from Low Complexity, Medium Complexity to High Complexity. The detailed evaluation of the complexity is as follows:

Rating Inputs:

- User registration details (email, password, confirmation, verification code, error message, confirmation message)
- Login credentials (email, password, error message)
- Biological metrics and fitness goal (age, weight, height, sex, activity level, goal, error message, confirmation message)
- Health data import (export file, source selection, activity type, duration, distance, heart rate,

timestamp, import summary, error message)

- Daily check-in (muscle fatigue, mood and yesterday's diet adherence, each rated 1–5; Singapore calendar date). Sleep hours are recorded separately for training readiness.
- Training preferences (training days per week, session duration, activity types, rest days, error message, confirmation message)
- Workout session logging (activity type, volume, duration, perceived exertion, heart rate, date, error message, confirmation message)
- Meal photo upload and confirmation (image, identified food items, portion sizes, calories, protein, carbohydrate, fat, confidence score, corrections)
- Friend request and removal (search query, selected user, confirmation)
- Chatbot query submission (query text, session identifier)
- Planned session reschedule (session identifier, target date, redistribution option, confirmation)

Files Type Referenced (FTR)	**Data Elements**		
	1-4	5-15	Greater than 15
Less than 2	Low (3)	Low (3)	Average (4)
2	Low (3)	Average (4)	High (6)
Greater than 2	Average (4)	High (6)	High (6)

Rating Outputs:

- **Displaying the dashboard training and nutrition trend charts (training volume, heart rate trend, readiness trend, distance covered, calories, macronutrient adherence, period selection)**
- **Displaying the calculated BMR, TDEE and daily macronutrient targets**
- **Displaying the daily readiness score with its intensity classification and contributing factors**
- **Displaying the generated multi-week training plan**
- **Displaying the daily recommended workout after readiness adjustment**
- **Displaying the estimated calories and macronutrients of an analysed meal**
- **Displaying the AI assistant response generated from the user's own records**
- **Displaying the leaderboard, achievements and experience points**

File Types Referenced (FTR)	**Data Elements**		
	1-5	6-19	Greater than 19
less than 2	Low (4)	Low (4)	Average (5)
2 or 3	Low (4)	Average (5)	High (7)

Greater than 3	Average (5)	High (7)	High (7)

Rating Inquiries:

- Selecting the training schedule for a chosen week or month
- Selecting the user's friend list
- Selecting the meal log for a chosen period
- Selecting the user's profile and settings
- Selecting past chat history
- Searching for another user by username

File Types Referenced (FTR)	**Data Elements**		
	1-5	6-19	Greater than 19
less than 2	Low (3)	Low (3)	Average (4)
2 or 3	Low (3)	Average (4)	High (6)
Greater than 3	Average (4)	High (6)	High (6)

Rating Logical Files:

- User Account and Profile
- Activity and Workout Session records
- Training Plan
- Check-in and Readiness records
- Meal and Nutrition records
- Social and Gamification records
- Chat history

Record Element Types (RET)	**Data Elements**		
	1 to 19	20 - 50	51 or More
1 RET	Low (7)	Low(7)	Average (10)
2 to 5 RET	Low (7)	Average (10)	High (15)
6 or More RET	Average (10)	High (15)	High (15)

Rating Interfaces:

- Google Gemini API (meal image recognition and conversational assistant)
- Strava API (activity data retrieved for display only)
- Supabase authentication service (user identity)

Record Element Types (RET)	Data Elements		
	1 to 19	20 - 50	51 or More
1 RET	Low (7)	Low(7)	Average (10)
2 to 5 RET	Low (7)	Average (10)	High (15)
6 or More RET	Average (10)	High (15)	High (15)

Summary of the above analysis:

Element	Complexity	Detail
Inputs	Low	User registration details
	Low	Login credentials
	Low	Biological metrics and fitness goal
	Low	Daily check-in
	Medium	Training preferences
	Medium	Workout session logging
	Medium	Meal photo upload and confirmation
	Low	Friend request and removal
	Low	Chatbot query submission
	Low	Planned session reschedule
Outputs	Medium	Dashboard training and nutrition trend charts
	Low	BMR, TDEE and macronutrient targets
	Medium	Daily readiness score and explanation
	High	Generated training plan
	Medium	Daily recommended workout
	Medium	Meal analysis estimate
	Medium	AI assistant response
	Low	Leaderboard, achievements and experience points
Inquiries	Low	Training schedule for a chosen week or month

	Low	Friend list
	Low	Meal log for a chosen period
	Low	Profile and settings
	Low	Past chat history
	Low	User search by username
Logical Files	Low	User Account and Profile
	Low	Activity and Workout Session records
	Medium	Training Plan
	Low	Check-in and Readiness records
	Medium	Meal and Nutrition records
	Low	Social and Gamification records
	Low	Chat history
Interfaces	Low	Google Gemini API
	Low	Strava API
	Low	Supabase authentication service

Calculation of Unadjusted Function Points:

Characteristic	Low		Medium		High	
	Score	Multiplier	Score	Multiplier	Score	Multiplier
Inputs	8	× 3	2	× 4	0	× 6
Outputs	2	× 4	5	× 5	1	× 7
Inquiries	6	× 3	0	× 4	0	× 6
Logical Files	5	× 7	2	× 10	0	× 15
Interfaces	3	× 5	0	× 7	0	× 10
Unadjusted FP	100		53		7	
Total=L+M+H	160					

5.1.2 Adjusted Function Points

Influence Factors	Score	Detail
Data Communications	5	The application is more than a front-end. It communicates over REST with its backend service, over OAuth 2.0 with Strava, and over HTTPS with the Gemini API, so more than one type of teleprocessing communications protocol is supported.
Distributed Functions	4	Online data transfer connects the Next.js application running on Node.js, one Python FastAPI backend and managed Supabase. FastAPI calls Gemini and Strava as external APIs. Docker Compose also provisions Redis, but no active API queue producer or worker consumer uses it.
Performance	3	Response time is critical during all business hours. The response-time targets applied are those defined in the Quality Plan. No special design for CPU utilisation was required.
Heavily used configuration	2	Some security and timing considerations are included. The application runs on shared hosting with no dedicated operational constraints.
Transaction rate	3	A daily peak transaction period is anticipated, as meal

		logging clusters around meal times and check-ins around the start of the day.
On-line data entry	5	More than 30% of transactions are interactive data entry, including check-ins, session logging, meal entry and profile updates.
End-user efficiency	4	Six or more of the efficiency designs are included: pre-filled forms, daily check-in questionnaire, drag-and-drop rescheduling, inline validation, photograph capture for meal entry, and a mobile-responsive layout.
Online data update	3	Online update of major internal logical files is included, covering profile, training plan, session and meal records.
Complex processing	4	Extensive logical and mathematical processing is required: readiness model inference, TDEE and macronutrient calculation using the Mifflin-St Jeor equation, periodisation with a safe volume threshold, and vision-based meal estimation.
Reusability	3	The application is documented and separates frontend UI, backend feature rules, readiness scoring and external integrations into modules. Application and inference modules share one FastAPI service and can be tested and maintained independently.
Installation Ease	1	No special considerations were stated by the user, but special setup is required for deployment and for configuring the external API credentials.
Operational Ease	2	Effective start-up, backup, and recovery processes are provided through the hosting platform, and no operator intervention is required.
Multiple sites	1	The application is designed for a single installation site, although it is accessible from desktop, laptop and mobile browsers.
Facilitate change	3	A flexible query and report facility is provided that can handle complex requests, including period filters and correlation views on one or more internal logical files.
Total score	**43**	

Influence Multiplier	**= Total score × 0.01 + 0.65 = 43 × 0.01 + 0.65 = 1.08**
Adjusted FP	**= Unadjusted FP × Influence Multiplier =** 160 × 1.08 = 172.80

Scoring (0 – 5)
0 = No influence
1 = Insignificant influence
2 = Moderate influence
3 = Average influence
4 = Significant influence
5 = Strong influence

5.1.3 Lines of Code

The Next.js application uses TypeScript and JavaScript and runs on Node.js. The backend uses Python FastAPI for application APIs, readiness inference and external integrations.

Based on the Capers Jones programming languages table [1], which expresses each language as a level and derives source statements per function point as 320 divided by that level, JavaScript sits at level 4.5 and therefore 320 / 4.5 = 71.11 LOC per FP. Python sits at level 6.0 and therefore 320 / 6.0 = 53.33 LOC per FP.

Approximately three quarters of the codebase is written in TypeScript or JavaScript, covering the Next.js frontend and the Node.js backend service, and one quarter in Python for the FastAPI inference service. The weighted average is therefore (0.75 x 71.11) + (0.25 x 53.33) = 53.33 + 13.33 = 66.67 LOC per FP.

Therefore, we have: Lines of code = 172.80 FP x 66.67 LOC / FP = **11521 LOC**.

5.2 Efforts, Duration and Team Size Estimation

To estimate the effort and duration required for the project, we use function points as the basis to calculate Effort, Duration, Team size and finally the schedule. The estimates are expanded to account for project management and extra contingency time to obtain the total average effort estimates. From these averages, the duration of each work package in working days is estimated based on the following calculations.

The project spans the school term from week 3 to week 12. One of those weeks is recess week, leaving 9 weeks of actual development effort.

Each team member commits 30 hours per week to the project. Across 9 weeks and 6 members, the available capacity is 30 x 9 x 6 = 1,620 person-hours. The estimate below requires 1,612.80 person-hours, or 268.8 hours per member, which is 29.9 hours per member per week and leaves 7.20 person-hours of headroom against capacity. The production rate of 60 LOC per person-day is higher than the 39 carried in the template because the project is greenfield, with no legacy code, no backwards compatibility and no external compliance review, and because the Supabase authentication, storage and row-level security layers and the Next.js and FastAPI scaffolding are configured rather than written. The plan fits the term, with no slack to spare.

- Working days include 5 days in a week.
- Effort = Size / Production Rate = (11521 LOC) / (60 LOC/PD) = 192.0 PD
- Duration = 3 x (Effort)^(1/3) = 3 x (192.0)^(1/3) = 17.3 Days
- Final development schedule = 17.3 Days / 5 days a week = 3.5 Weeks

the 3.5-week figure is the nominal schedule the formula gives at full-time staffing; at the team's actual 30 hours per week it works out to 268.8 / 30 = 9.0 weeks per member, which is exactly the 9 weeks available.

5.2.1 Distribution of Effort

1990s Industry Data	Work Package	Distribution	Estimates (PH)
Preliminary Design 18 %	Project Plan	9%	138.24
	Requirement Specification	9%	138.24
Detailed Design 25 %	User Interface	7%	107.52
	Technical Architecture	11%	168.96
	Data Modeling	7%	107.52
Code & Unit Testing 26 %	Code & Unit Testing	21%	322.56
	Online Documentation	5%	76.80
Integration & Test 31 %	Integration & Quality Assurance	31%	476.16
	Extrapolated total effort		**1536.00**

	2% for project management		**30.72**
	3% for contingency		**46.08**
	Total effort		**1612.80**

These duration estimates are based on the assumption that each team member works an equal amount on any given work package.

- Individual effort = 1612.80 / 6 members = 268.8 H each
- 268.8 H / 30 H per week = 9.0 weeks

5.3 Cost Estimates

The project incurs no licensing cost, as every external service and development tool used is available on a free tier that is sufficient for the expected demonstration workload.

API and Infrastructure Subscriptions:

Service	**Plan Used**	**Cost**
Google Gemini API	Free tier	$0
Supabase (database and authentication)	Free tier	$0
Strava API	Free developer access	$0
Vercel (application hosting)	Hobby tier	$0
GitHub (version control and CI)	Free for public repositories	$0
Jira (backlog and defect tracking)	Free plan, up to 10 users	$0
Total		**$0**

Standard tier access to the Strava API requires an active Strava subscription. Should the team exceed the free developer allowance, this would add approximately $11.99 per month for one account. Exceeding the Gemini or Supabase free tier quotas would likewise move those services onto a paid plan. These are identified as cost risks in the risk management section rather than as committed costs.

Manpower:

The total estimated effort of 1,612.80 person-hours is met entirely by the six project team members and carries no monetary cost to the project. At a notional graduate engineer rate of $25 per hour, the equivalent commercial cost of this effort would be approximately $40,320.

6 Product Checklist

The plan is that the items listed below will be delivered on the stated deadlines.

Task	Estimated Deadline
Lab 1 Tasks	
Team Information	27 August 2026
Project Proposal	10 September 2026
Use Case Model	10 September 2026
Lab 2 Tasks	
System Requirements Specification (SRS)	24 September 2026
Quality Plan	24 September 2026
Lab 3 Tasks	
Project Plan	15 October 2026
Risk Management	15 October 2026
Demo (Preparation)	15 October 2026
Github	15 October 2026
Lab 4 Tasks	

Demo	15 October 2026
Design Report on Software Maintainability	29 October 2026
Configuration Management Plan	29 October 2026
Change Management Plan	29 October 2026
Release Plan	29 October 2026
Lab 5 Task	
Presentation	29 October 2026
Presentation Slides	29 October 2026
Test Plan	29 October 2026
Test Cases and Requirements Test Coverage Report	29 October 2026

7 Best Practice Checklist

Practice	Score
We must standardise all our documentation formats.	
The team must actively scrutinise every single requirement to guarantee absolute accuracy, completeness, and consistency while eliminating any hidden ambiguities. Include a complete functional specification.	
Complexity management remains one of our most significant daily challenges. Strive to: • Minimise interfaces between modules, procedures, and data structures • Limit personnel interfaces to avoid exponential communication costs • Design strictly to meet actual customer requirements without ever adding unnecessary or overly complicated product functions.	
We demand continuous project visibility to accurately measure actual progress and take immediate management action when necessary. Managers must maintain excellent communication with their employees. Submit all code for peer evaluation, and evaluate every proposed design for strict technical appropriateness.	
We plan for continuous change. We must: • Explicitly stamp all manuals, designs, tests, and source code with clear revision numbers, dates, history comments, and distinct change marks. • Ensure managers approve new revisions before implementation, then check these updates for strict quality and compliance. • Use a formalised configuration management system to enforce required maintenance	
Never underestimate project variables. We must obtain highly accurate estimates covering time, raw effort, administrative overhead, and meeting duration. Specifically, budget adequate effort for the gruelling phases of system integration, rigorous software testing, comprehensive documentation, and long-term platform maintenance.	
Code reviews expose software defects far more efficiently than standard testing, so we actively plan and manage these peer evaluations across the team.	
Execute comprehensive software testing using both black-box and white-box methodologies to evaluate the platform. We enforce dedicated unit, functional, integration, and user acceptance testing.	

8 Risk Management

Besides the general risk management, the following risks have been identified for the Fitness Tracker application:

Third-party terms of service restrict a planned feature	
Impact Severity	High
Probability	40%
Impacts	A feature already designed and documented might have to be redesigned or even cancelled.
Risk Reduction	Check the current terms of service of any external service used before designing a feature around it. Record the restriction as a business rule in the affected use cases so it is not reversed later by a developer who is unaware of it.

Free-tier API quota exhausted during development or demonstration	
Impact Severity	High
Probability	30%
Impacts	The Gemini and Supabase free tiers limit request rates and storage counts, resulting in a possibility of exhausting the quota. Exhausting the quota would prevent the AI components from running during user demonstrations.
Risk Reduction	Monitor usage against quota at each sprint review. Cache inference results during testing rather than issuing a fresh call each time. Prepare seeded demonstration data and a recorded fallback so the demo can proceed if a live call fails.

Meal image recognition accuracy below usable threshold	
Impact Severity	Moderate
Probability	35%
Impacts	Users receive wrong calorie and macronutrient estimates, which undermines the nutrition tracking and the dashboard analytics built on it.
Risk Reduction	Require user confirmation of every estimate before it is saved. Set a confidence threshold below which the system falls back to manual entry rather than storing a poor estimate.

Strava policy breach or data leakage into AI	
Impact Severity	High
Probability	5%
Impacts	Strava strictly prohibits feeding their telemetry into external artificial intelligence models. If our backend accidentally leaks this restricted user telemetry into Gemini prompts, Strava will immediately and permanently revoke our developer API access. Losing this access severely cripples the core application.
Risk Reduction	Keep a display-only data path, exclude Strava fields from AI schemas and add tests at each outbound AI boundary.

Customer cancels project	
Impact Severity	Critical
Probability	1%

Impacts	Cancellation wastes all development work and project resources
Risk Reduction	We maintain weekly contact with the customer

9 Quality Assurance

Our team ensures high software quality by following the strict assessment standards defined in the Software Quality Assurance Plan. We document all specific procedures inside the formal Test Plan. This guarantees accountability. To maintain code integrity, our developers use distinct methodologies that catch defects early and enforce strict coverage targets:

- **Unit Testing:**
  Developers test individual components continuously. We require automated unit test execution on every single pull request before anyone can merge code. This strict rule helps the team hit minimum coverage thresholds of eighty percent for code lines and seventy-five percent for branches
- **In-Place Testing:**
  We evaluate the entire application as a complete unit. The QA team runs targeted validation sequences. These sequences verify that all customer requirements operate exactly as intended within the actual production environment. These distinct methodologies target two critical aspects of the Smart Fitness Tracker.
- **System Function:**
  We test the platform extensively to eliminate software flaws. The team monitors performance metrics closely. This ensures that internal database endpoints and external API calls reply quickly and securely under load.
- **Algorithmic Function:**
  We validate complex system mathematics directly. Calculations matter. QA personnel assess the Mifflin-St Jeor equation calculations and ensure the training generation logic consistently enforces progressive overload safety constraints.

To ensure absolute reliability, the QA team actively builds vast suites of realistic test cases based directly on our documented system requirements. We validate the code daily. While real athletes will eventually supply actual telemetry data, our engineering team creates highly detailed mockup user profiles for all internal testing purposes. We test edge cases constantly. By deliberately injecting invalid scenarios like negative user weights, missing biometric fields, or failing AI vision confidence scores, we guarantee the application always recovers gracefully from unexpected user inputs.

10 Monitoring & Control

Many procedures are required in order to be able to successfully monitor the progress of a software project. Some of the most important are:

Quantitative measurement of resource consumption:

We must track progress accurately against our nine-week milestone schedule. By estimating the development team's resource requirements, specifically available hours and task allocation, we gain a clear quantitative measurement of our true progress. The team updates a Burndown chart regularly. This visualises completed tasks against the remaining backlog to keep everyone aligned.

Identification of major project risks:

Identifying major risks early prevents compounding failures. For instance, we actively monitor third-party API rate limits for Strava and Gemini, alongside potential database connection issues, so we can implement preventative measures immediately. The Risk Management section details these critical threats and outlines specific fallback strategies to avoid them. We conduct milestone-driven risk reviews to maintain strict stability.

Regular reviews of project progress:

The development team holds weekly Sprint meetings. During these sessions, members review the progress of management, planning, analysis, prototype development, and testing tasks. If anyone spots a significant deviation that threatens the delivery timeline, they immediately flag it for the project manager to take corrective action. Open communication keeps the project on track.

Timeline Planning and task decomposition:

This document defines a clear timeline from project commencement through the final Lab 5 presentation. We built this reasonably accurate schedule by hierarchically decomposing large tasks into measurable subcomponents within Jira and our project backlog. This decomposition simplifies task assignment. Managers distribute work across frontend, backend, and QA roles, enabling fine-grained progress measurement throughout the entire implementation phase.

11 References

1. greg, "#12 – EARLY SIZING and EARLY RISK ANALYSIS for SOFTWARE PROJECTS – (C) CAPERS JONES | CERM ® RISK INSIGHTS," *Cermacademy.com*, Apr. 28, 2013. https://insights.cermacademy.com/12-early-sizing-and-early-risk-analysis-for-software-projects-c-capers-jones/